

- Document 06: Next.js Edge Frontend Architecture
 - CodePerfect Auditor — React Server Components, State Management & UI Design
 - Overview
 - Application Route Structure
 - Explanation
 - The Next.js Middleware — src/middleware.ts
 - Guard 1: Unauthenticated Access Prevention
 - Guard 2: Smart Post-Login Redirect
 - Guard 3: Route-Level RBAC Enforcement
 - Explanation
 - The Global Layout — src/app/layout.tsx
 - Pydantic Data Contracts — backend/models.py
 - Clinical Extraction Models
 - API Request & Response Models
 - Analytics Models
 - Explanation
 - Analytics API — Real-Time KPI Computation
 - Explanation
 - Case History API — Paginated Joins
 - Frontend Portal Design Summary

Document 06: Next.js Edge Frontend Architecture

CodePerfect Auditor — React Server Components, State Management & UI Design

Project: CodePerfect Auditor | **Version:** 1.0 | **Date:** 31-03-2026 **Submitted To:** Virtusa Hackathon | **Institution:** Saveetha Engineering College

Overview

The CodePerfect Auditor frontend is built on **Next.js 14** using the **App Router**, TypeScript, and the **React Server Components (RSC)** model. It is a dual-portal enterprise web application serving two completely separate user experiences on the same codebase:

- 1. **Hospital Portal** (`/hospital/*`) — Coding analysts submit clinical charts, receive AI-generated ICD codes, and manage claims.
- 2. **Payer Portal** (`/payer/*`) — Insurance adjudicators review submitted claims and make reimbursement decisions.

The architecture enforces strict separation between these portals at three levels: **URL structure**, **Next.js Middleware RBAC**, and **React layout-level role guards**.

Application Route Structure

```
src/app/
├── page.tsx           ← Public landing page
├── layout.tsx         ← Global HTML shell + font loading
├── globals.css        ← Design system tokens and global styles
├── auth/
│   ├── login/page.tsx ← Supabase Auth sign-in form
│   └── signup/page.tsx ← Organization registration flow
├── hospital/
│   ├── layout.tsx      ← Hospital shell (sidebar + RBAC guard)
│   ├── coder/
│   │   ├── analyze/page.tsx ← Main AI coding workspace
│   │   └── history/page.tsx ← Case history table
│   ├── rcm/
│   │   ├── inbox/page.tsx  ← Claims inbox (RCM staff view)
│   │   └── analytics/page.tsx ← Revenue analytics dashboard
│   └── admin/
│       ├── branches/page.tsx ← Branch management (admin only)
│       └── users/page.tsx    ← User management (admin only)
├── payer/
│   ├── layout.tsx      ← Payer shell (sidebar + RBAC guard)
│   ├── inbox/page.tsx  ← Payer claims queue
│   └── adjudicate/[id]/page.tsx ← Individual claim review
```

Explanation

The file-based routing in Next.js App Router means every directory maps to a URL segment. The nested `layout.tsx` files are the key architectural element — they wrap every child page with the sidebar, authentication guard, and RBAC enforcement. A `coder` user navigating to `/hospital/admin/users` hits the layout guard before the page ever renders, gets redirected to `/403`, and the admin `users/page.tsx` component is never even loaded.

The Next.js Middleware — `src/middleware.ts`

The middleware file is the **first execution layer** — it runs on Cloudflare/Vercel's Edge Runtime before any React component, before any database query, and before any page HTML is sent to the browser.

```
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const response = NextResponse.next({ request });

  // Create a Supabase server client that reads/writes session cookies.
  // This is the @supabase/ssr package – NOT the browser client.
  // It handles cookie-based JWT refresh automatically on every request.
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() { return request.cookies.getAll(); },
        setAll(cookiesToSet) {
          // Write updated session cookies to BOTH the request and
          // to prevent a stale session from being served on the next
          response

          cookiesToSet.forEach(({ name, value, options }) => {
            request.cookies.set(name, value);
```

```

        response.cookies.set(name, value, options);
    });
    },
},
}
);

const { data: { session } } = await supabase.auth.getSession();
const isLoggedIn = !!session;

```

Guard 1: Unauthenticated Access Prevention

```

// Any route under /hospital/* or /payer/* requires authentication.
const isProtectedPath = pathname.startsWith('/hospital') ||
pathname.startsWith('/payer');
if (isProtectedPath && !isLoggedIn) {
    // Hard redirect to login – no flicker, no flash of protected content.
    // This runs at the Edge, before any React rendering begins.
    return NextResponse.redirect(new URL('/auth/login', request.url));
}

```

Guard 2: Smart Post-Login Redirect

```

// If a logged-in user visits /auth/login or /auth/signup,
// redirect them to the correct portal based on their org type and role.
if (pathname.startsWith('/auth') && isLoggedIn && session?.user?.id) {
    const { data: userRow } = await supabase
        .from('users')
        .select('role, organizations(type)')
        .eq('auth_id', session.user.id)
        .single();

    const role = userRow?.role as string | null;
    const orgType = (userRow?.organizations as { type?: string } | null)?.type;

    if (orgType === 'insurance_payer') {
        // Payer organizations → payer inbox
        return NextResponse.redirect(new URL('/payer/inbox', request.url));
    } else if (role === 'rcm') {
        // RCM staff → claims inbox
        return NextResponse.redirect(new URL('/hospital/rcm/inbox',
request.url));
    } else {
        // Coders, admins, auditors → coding workspace
        return NextResponse.redirect(new URL('/hospital/coder/analyze',
request.url));
    }
}

```

```
}  
}
```

Guard 3: Route-Level RBAC Enforcement

```
if (isProtectedPath && isLoggedIn && session?.user?.id) {  
  const { data: userRow } = await supabase  
    .from('users')  
    .select('role, organizations(type)')  
    .eq('auth_id', session.user.id)  
    .single();  
  
  const role = userRow?.role as string | null;  
  const orgType = (userRow?.organizations as { type?: string } | null)?.type;  
  
  // Hard cross-portal isolation:  
  // A payer adjudicator can NEVER access /hospital/* routes  
  if (pathname.startsWith('/hospital') && orgType === 'insurance_payer') {  
    return NextResponse.redirect(new URL('/payer/inbox', request.url));  
  }  
  // A hospital coder can NEVER access /payer/* routes  
  if (pathname.startsWith('/payer') && orgType !== 'insurance_payer') {  
    return NextResponse.redirect(new URL('/hospital/coder/analyze',  
request.url));  
  }  
  
  // Within the hospital portal – sub-route RBAC:  
  // /hospital/coder/* → only coder or admin  
  if (pathname.startsWith('/hospital/coder') && ![ 'coder',  
'admin' ].includes(role)) {  
    return NextResponse.redirect(new URL('/403', request.url));  
  }  
  // /hospital/rcm/* → only rcm or admin  
  if (pathname.startsWith('/hospital/rcm') && ![ 'rcm',  
'admin' ].includes(role)) {  
    return NextResponse.redirect(new URL('/403', request.url));  
  }  
  // /hospital/admin/* → admin only  
  if (pathname.startsWith('/hospital/admin') && role !== 'admin') {  
    return NextResponse.redirect(new URL('/403', request.url));  
  }  
  // /payer/automation → payer admin only  
  if (pathname.startsWith('/payer/automation') && role !== 'admin') {  
    return NextResponse.redirect(new URL('/payer/inbox', request.url));  
  }  
}  
  
// Matcher: run middleware on ALL routes except static files  
export const config = {  
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\.
```

```
(?:svg|png|jpg|jpeg|gif|webp)$).*')'],  
};
```

Explanation

Middleware runs on Vercel's **Edge Runtime** — not in Node.js. This means RBAC enforcement has near-zero latency regardless of traffic load. The **matcher** configuration excludes static asset files (**.js**, **.css**, **.png**) from middleware processing, which is a critical performance optimization — without it, every image load would trigger a Supabase JWT verification round-trip.

The Global Layout — **src/app/layout.tsx**

```
// The root layout wraps every page in the application.  
// It handles: HTML document structure, global fonts, and the AuthProvider context.  
export const metadata: Metadata = {  
  title: 'CodePerfect Auditor',  
  description: 'AI-powered medical coding and revenue integrity platform',  
};  
  
export default function RootLayout({ children }: { children: React.ReactNode }) {  
  return (  
    <html lang="en">  
      <body className={inter.className}>  
        <AuthProvider>    {/* Global auth context – all pages read from  
here */}  
          {children}  
        </AuthProvider>  
      </body>  
    </html>  
  );  
}
```

Pydantic Data Contracts — **backend/models.py**

The strict Pydantic models define the data contract between the FastAPI backend and the Next.js frontend. TypeScript on the frontend mirrors these models.

Clinical Extraction Models

```
class DiagnosisEntity(BaseModel):
    """One diagnosis extracted by the LLM from the clinical text."""
    text: str # e.g. "Type 2 Diabetes with CKD stage 3"
    severity: Optional[str] = None # "mild" | "moderate" | "severe"
    laterality: Optional[str] = None # "left" | "right" | "bilateral"
    snomed_candidate: SnomedCandidate # AI's initial SNOMED guess before Node 3
    comorbidities: List[str] = [] # Related conditions mentioned in the same chart
    evidence_text: str # The exact sentence that supports this diagnosis

    @field_validator("severity", "laterality", mode="before")
    @classmethod
    def sanitize_null_fields(cls, v):
        """LLMs sometimes return the string 'null' instead of JSON null. Fix it."""
        if isinstance(v, str) and v.strip().lower() in ("null", "none", ""):
            return None
        return v

class ExtractionResult(BaseModel):
    """The complete output of the clinical NLP extraction node (Node 2)."""
    patient: Optional[PatientDemographics] = None
    diagnoses: List[DiagnosisEntity] # The primary output
    observations: List[ObservationEntity] = [] # Vitals, lab values
    procedures_and_services: List[str] = [] # Extracted CPT procedure text
```

API Request & Response Models

```
class CodeRequest(BaseModel):
    """Request body for POST /api/v1/code/run – the main pipeline trigger."""
    raw_text: str # Clinical notes text (mandatory)
    session_id: Optional[str] = None # If None, pipeline generates a UUID
    human_icd_code: Optional[str] = None # For audit comparison (Node 7)
    org_id: Optional[str] = None # For CPT pricing multiplier lookup

class CodeResponse(BaseModel):
    """Response from POST /api/v1/code/run – the complete pipeline output."""
    session_id: str
    final_icd_code: str # e.g. "E11.22"
    confidence_score: float # 0.0 – 1.0
```

```

mapping_path: str # "direct" | "embedding" |
"who_api_icd11"
resolved_snomed_code: Optional[str] # e.g. "73211009"
icd_codes: List[dict] # Primary + secondary + additional codes
cpt_codes: List[dict] # CPT codes with gross charges applied
discrepancy_type: Optional[str] # EXACT_MATCH | OVERCODING | etc.
financial_delta: Optional[float] # Revenue difference (AI vs human code)
drg_flag: Optional[str] # MCC_MISSED | CC_MISSED | None
risk_score: float
risk_label: str # LOW | MEDIUM | HIGH
financial_summary: Optional[dict] # {total_estimated_revenue,
pricing_multiplier, line_items}
fhir_condition: Optional[dict] # FHIR R4 Condition resource
patient_name: Optional[str]
patient_dob: Optional[str]
ocr_used: Optional[bool] # True if Tesseract was triggered on a
scanned PDF

```

Analytics Models

```

class AnalyticsOverview(BaseModel):
    """KPI dashboard payload – powers the 4 summary cards + 30-day trend chart."""
    total_cases: int
    total_revenue_recovered: float
    avg_confidence: float # 0-100 (percentage)
    high_risk_rate: float # 0-100 (percentage)
    risk_distribution: dict # {"LOW": 47, "MEDIUM": 23, "HIGH": 8}
    source_distribution: dict # {"text_input": 52, "pdf_upload": 26}
    trend: List[TrendPoint] # 30 data points (last 30 days)

class CaseSummary(BaseModel):
    """One row in the paginated Case History table."""
    result_id: str
    session_id: str
    ai_icd_code: Optional[str]
    human_icd_code: Optional[str]
    discrepancy_type: Optional[str]
    financial_delta: Optional[float]
    risk_score: float
    risk_label: str
    confidence_score: float
    drg_flag: Optional[str]
    document_source: Optional[str] # "text_input" | "pdf_upload"
    ocr_used: Optional[bool]
    text_snippet: Optional[str] # First 300 chars for preview in table
    created_at: str

```

Explanation

The `@field_validator` with `sanitize_null_fields` is a production-grade defensive measure specific to medical LLM outputs. When Groq `llama-3.3-70b` extracts clinical entities and a field has no value, it sometimes returns the raw string `"null"` or `"None"` instead of a proper JSON `null`. Without this validator, `"null"` would be stored in `severity` — corrupting the audit log and causing TypeScript type errors in the frontend when the string `"null"` is passed to components expecting `null | undefined`.

Analytics API — Real-Time KPI Computation

```
# routes/analytics.py
@router.get("/overview", response_model=AnalyticsOverview)
async def get_analytics_overview():
    """
    Computes KPIs from raw coding_results rows – no stored procedures or views.
    All aggregation is done in Python for maximum transparency and testability.
    """
    rows = await select(
        "coding_results",

query="risk_label,discrepancy_type,financial_delta,confidence_score,created_at,"
        "clinical_cases(document_source)",
    )

    risk_dist = {"LOW": 0, "MEDIUM": 0, "HIGH": 0}
    daily_cases = defaultdict(int)
    daily_revenue = defaultdict(float)

    for r in rows:
        label = r.get("risk_label", "LOW")
        delta = float(r.get("financial_delta") or 0)
        if label in risk_dist:
            risk_dist[label] += 1
        if delta > 0:
            revenue_sum += delta

    # 30-day trend bucketing
    created = datetime.fromisoformat(r["created_at"].replace("Z", "+00:00"))
    if created >= cutoff:
        day_key = created.strftime("%Y-%m-%d")
        daily_cases[day_key] += 1
        daily_revenue[day_key] += delta

    # Generate a complete 30-day array (filling zero for days with no cases)
    trend = [
        TrendPoint(
            date=day,
```

```

        cases=daily_cases.get(day, 0),
        revenue=round(daily_revenue.get(day, 0.0), 2),
    )
    for i in range(29, -1, -1)
    for day in [(now - timedelta(days=i)).strftime("%Y-%m-%d")]
]

```

Explanation

The 30-day trend array always has exactly 30 data points — even if days have no cases (they get `cases: 0, revenue: 0.0`). This is a deliberate UX engineering decision: a frontend chart library receiving a sparse array of only active days would render with gaps and inconsistent X-axis spacing, looking broken to the user. A complete 30-point array always renders a clean, consistent chart.

Case History API — Paginated Joins

```

# routes/cases.py
@router.get("", response_model=CaseListResponse)
async def list_cases(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    risk_label: Optional[str] = Query(None), # filter: LOW | MEDIUM | HIGH
    document_source: Optional[str] = Query(None), # filter: text_input |
pdf_upload
    branch_id: Optional[str] = Query(None),
):
    """
    Uses select_paginated() to perform a range-limited PostgREST query.
    The embedded join syntax fetches coding_results + clinical_cases in one call:
    coding_results!inner(session_id, document_source, ocr_used, raw_text_snippet)
    """
    rows, total = await select_paginated(
        table="coding_results",
        query="result_id,ai_icd_code,risk_score,risk_label,created_at,"

"clinical_cases!inner(session_id,document_source,ocr_used,raw_text_snippet)",
        order="created_at.desc", # Most recent cases first
        page=page,
        page_size=page_size,
    )
    return CaseListResponse(
        cases=summaries,
        total=total,
        page=page,

```

```
    page_size=page_size,  
    total_pages=max(1, -(-total // page_size)), # Ceiling integer division  
)
```

Frontend Portal Design Summary

Feature	Implementation
Framework	Next.js 14, App Router, TypeScript
Auth	Supabase SSR (@supabase/ssr) — cookie-based JWT refresh
Middleware	Edge Runtime — runs before any React render
RBAC	3-layer: Middleware → Layout guard → DOM null returns
Data Fetching	fetch() to FastAPI REST endpoints behind /api/v1
Styling	Vanilla CSS + CSS Variables design tokens + globals.css
Components	Lucide React icons, custom glassmorphism cards
PDF Upload	multipart/form-data → FastAPI /code/run-pdf
Real-time	Polling on claim status (payer inbox)
Build	next build → Static + Server rendering (ISR)